SAVITRIBAI PHULE PUNE UNIVERSITY • NEP 2020 PATTERN (2025–2026)

S.Y. B.Sc. (CA) / BCA (Science) Sem-III — Dev-C++ Practical Manual

Courses: CA-202-MJP (Data Structure Lab) & ELS-242-MNP (Data Comm. Lab) **Compiler Standard:** GCC / MinGW (ISO C99/C11)

Feature / Construct	Legacy Turbo C (16-Bit DOS) — Avoid	Modern Dev-C++ (GCC / MinGW) — Used Here
Header Files	`#include <conio.h>`, `<alloc.h>`	`#include <stdio.h>`, `<stdlib.h>`, `<string.h>`, `<ctype.h>`
Entry Function	`void main()` (Non-standard)	`int main()` ending with `return 0;` (Standard ANSI/ISO C)
Screen & Pause	`clrscr();` and `getch();`	Not required; Dev-C++ console pauses automatically after execution
Character Input	`gets(str)` / unbuffered `scanf("%c")`	`scanf("%99s", str)` and leading space `scanf(" %c", &ch)`

SLIP 1 — Binary Search & Static Stack Implementation

Max Marks: 30 (Q1: 10 | Q2: 15 | Viva: 05)

Q.1) Binary Search in a Sorted Array (10M)

```
#include <stdio.h>

int main() {
    int a[50], n, i, key, low, high, mid;
    printf("Enter n: ");
    scanf("%d", &n);
    printf("Enter %d sorted elements:\n",
n);
    for (i = 0; i < n; i++) scanf("%d",
&a[i]);
    printf("Enter key to search: ");
    scanf("%d", &key);

    low = 0; high = n - 1;
    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid] == key) {
            printf("Found %d at index %d
(Pos %d)\n",
                   key, mid, mid + 1);
            return 0;
        } else if (a[mid] < key) low =
mid + 1;
        else high = mid - 1;
    }
    printf("Element %d not found.\n",
key);
    return 0;
}
```

Output: Enter n: 5 | Sorted: 10 20 30 40 50
Enter key: 30 → Found 30 at index 2 (Pos 3)

Q.2) Static Stack of Integers (Push, Pop, Display) (15M)

```
#include <stdio.h>
#define MAX 5
int st[MAX], top = -1;

void push() {
    int v;
    if (top == MAX - 1) printf("Stack
Overflow!\n");
    else {
        printf("Val: "); scanf("%d", &v);
        st[++top] = v;
    }
}
void pop() {
    if (top == -1) printf("Stack Underflow!\n");
    else printf("Popped: %d\n", st[top--]);
}
void display() {
    if (top == -1) printf("Stack Empty\n");
    else {
        for (int i = top; i >= 0; i--) printf("%d
", st[i]);
        printf("\n");
    }
}
int main() {
    int ch;
    do {
        printf("1.Push 2.Pop 3.Display 4.Exit: ");
        scanf("%d", &ch);
        if (ch == 1) push();
        else if (ch == 2) pop();
        else if (ch == 3) display();
    } while (ch != 4);
    return 0;
}
```

Output: 1.Push (10) → 1.Push (20) → 3.Display: 20 10 → 2.Pop: 20

SLIP 2 — Bubble Sort & Singly Linked List (Insert / Delete)

Max Marks: 30 (Q1: 10 | Q2: 15 | Viva: 05)

Q.1) Sort n Elements using Bubble Sort (10M)

```
#include <stdio.h>

int main() {
    int a[50], n, i, j, t;
    printf("Enter n: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) scanf("%d",
&a[i]);

    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                t = a[j];
                a[j] = a[j + 1];
                a[j + 1] = t;
            }
        }
    }
    printf("Sorted Ascending: ");
    for (i = 0; i < n; i++) printf("%d ",
a[i]);
    printf("\n");
    return 0;
}
```

Output: Enter n: 4 | Input: 45 12 78 23
Sorted Ascending: 12 23 45 78

Q.2) Singly Linked List (Insert, Delete, Display) (15M)

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node* next; } *head
= NULL;

void insert(int v) {
    struct Node* n = (struct
Node*)malloc(sizeof(struct Node));
    n->data = v; n->next = head; head = n;
}
void del(int v) {
    struct Node *t = head, *p = NULL;
    if (t && t->data == v) { head = t->next;
free(t); return; }
    while (t && t->data != v) { p = t; t = t-
>next; }
    if (!t) { printf("Not found\n"); return; }
    p->next = t->next; free(t);
}
void display() {
    for (struct Node* p = head; p; p = p->next)
        printf("%d -> ", p->data);
    printf("NULL\n");
}
int main() {
    insert(30); insert(20); insert(10); display();
    del(20); display();
    return 0;
}
```

Output: 10 -> 20 -> 30 -> NULL
After del(20): 10 -> 30 -> NULL

SLIP 3 — Insertion Sort & Linear Queue Operations

Max Marks: 30 (Q1: 10 | Q2: 15 | Viva: 05)

Q.1) Sort Numbers using Insertion Sort (10M)

```
#include <stdio.h>

int main() {
    int a[50], n, i, j, key;
    printf("Enter n: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) scanf("%d",
&a[i]);

    for (i = 1; i < n; i++) {
        key = a[i];
        j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
    printf("Sorted Array: ");
    for (i = 0; i < n; i++) printf("%d ",
a[i]);
    printf("\n");
    return 0;
}
```

Output: Enter n: 5 | Input: 34 12 52 9 85
Sorted Array: 9 12 34 52 85

Q.2) Queue Implementation (Insert, Delete, Display) (15M)

```
#include <stdio.h>
#define SIZE 5
int q[SIZE], f = -1, r = -1;

void insert(int v) {
    if (r == SIZE - 1) printf("Queue
Overflow!\n");
    else {
        if (f == -1) f = 0;
        q[++r] = v;
    }
}
void deleteQ() {
    if (f == -1 || f > r) printf("Queue
Underflow!\n");
    else {
        printf("Deleted: %d\n", q[f++]);
        if (f > r) f = r = -1;
    }
}
void display() {
    if (f == -1) printf("Empty\n");
    else {
        for (int i = f; i <= r; i++) printf("%d ",
q[i]);
        printf("\n");
    }
}
int main() {
    insert(10); insert(20); insert(30); display();
    deleteQ(); display();
    return 0;
}
```

SLIP 4 — Character Linear Search & Circular Queue

Max Marks: 30 (Q1: 10 | Q2: 15 | Viva: 05)

Q.1) Linear Search for a Character (10M)

```
#include <stdio.h>

int main() {
    int n, i;
    char a[50], key;
    printf("Enter number of chars: ");
    scanf("%d", &n);
    printf("Enter %d characters: ", n);
    for (i = 0; i < n; i++) scanf(" %c",
&a[i]);
    printf("Enter char to search: ");
    scanf(" %c", &key);

    for (i = 0; i < n; i++) {
        if (a[i] == key) {
            printf("Char '%c' at index %d
(Pos %d)\n",
                   key, i, i + 1);
            return 0;
        }
    }
    printf("Character '%c' not found.\n",
key);
    return 0;
}
```

Output: Chars: a e i o | Search: i
Char 'i' at index 2 (Pos 3)

Q.2) Circular Queue (Insert, Delete, Display) (15M)

```
#include <stdio.h>
#define SZ 5
int cq[SZ], f = -1, r = -1;

void ins(int v) {
    if ((r + 1) % SZ == f) { printf("Overflow\n");
return; }
    if (f == -1) f = r = 0;
    else r = (r + 1) % SZ;
    cq[r] = v;
}
void del() {
    if (f == -1) { printf("Underflow\n"); return;
}
    printf("Deleted: %d\n", cq[f]);
    if (f == r) f = r = -1;
    else f = (f + 1) % SZ;
}
void disp() {
    if (f == -1) return;
    for (int i = f; ; i = (i + 1) % SZ) {
        printf("%d ", cq[i]);
        if (i == r) break;
    }
    printf("\n");
}
int main() {
    ins(10); ins(20); ins(30); disp();
    del(); disp();
    return 0;
}
```

Output: 10 20 30 → Deleted: 10 → 20 30

SLIP 5 — Reverse String using Stack & Binary Search Tree (BST)

Max Marks: 30 (Q1: 10 | Q2: 15 | Viva: 05)

Q.1) Reverse a String using Stack (10M)

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100], st[100];
    int top = -1, i;
    printf("Enter a string: ");
    scanf("%99s", str);

    for (i = 0; str[i] != '\0'; i++) {
        st[++top] = str[i];
    }
    printf("Reversed string: ");
    while (top >= 0) {
        printf("%c", st[top--]);
    }
    printf("\n");
    return 0;
}
```

Output: Enter a string: Pune
Reversed string: enuP

Q.2) BST In-Order, Leaf & Non-Leaf Count (15M)

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *l, *r; };

struct Node* ins(struct Node* t, int v) {
    if (!t) {
        t = (struct Node*)malloc(sizeof(struct
Node));
        t->data = v; t->l = t->r = NULL; return t;
    }
    if (v < t->data) t->l = ins(t->l, v);
    else if (v > t->data) t->r = ins(t->r, v);
    return t;
}
void inorder(struct Node* t) {
    if (t) { inorder(t->l); printf("%d ", t-
>data); inorder(t->r); }
}
int leaf(struct Node* t) {
    if (!t) return 0;
    if (!t->l && !t->r) return 1;
    return leaf(t->l) + leaf(t->r);
}
int nonLeaf(struct Node* t) {
    if (!t || (!t->l && !t->r)) return 0;
    return 1 + nonLeaf(t->l) + nonLeaf(t->r);
}
int main() {
    struct Node* root = NULL;
    int a[] = {50, 30, 70, 20, 40};
    for (int i = 0; i < 5; i++) root = ins(root,
a[i]);
    printf("In-order: "); inorder(root);
```

SLIP 6 — Square of Elements Array & Menu-Driven Singly Linked List

Max Marks: 30 (Q1: 10 | Q2: 15 | Viva: 05)

Q.1) Store Square of Array Elements (10M)

```
#include <stdio.h>

int main() {
    int n, i, a[50], sq[50];
    printf("Enter n: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        sq[i] = a[i] * a[i];
    }
    printf("Original: ");
    for (i = 0; i < n; i++) printf("%d ",
a[i]);
    printf("\nSquares:  ");
    for (i = 0; i < n; i++) printf("%d ",
sq[i]);
    printf("\n");
    return 0;
}
```

Output:
```
Original: 2 4 6 8
Squares: 4 16 36 64
```

Q.2) Menu-Driven SLL (Create, Display, Delete) (15M)

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node* next; } *head
= NULL;

void create(int v) {
    struct Node* n = (struct
Node*)malloc(sizeof(struct Node));
    n->data = v; n->next = head; head = n;
}
void del(int v) {
    struct Node *t = head, *p = NULL;
    if (t && t->data == v) { head = t->next;
free(t); return; }
    while (t && t->data != v) { p = t; t = t-
>next; }
    if (t) { p->next = t->next; free(t); }
}
void disp() {
    for (struct Node* t = head; t; t = t->next)
        printf("%d -> ", t->data);
    printf("NULL\n");
}
int main() {
    int ch, v;
    do {
        printf("1.Create 2.Disp 3.Del 4.Exit: ");
        scanf("%d", &ch);
        if (ch == 1) { scanf("%d", &v); create(v);
}
        else if (ch == 2) disp();
        else if (ch == 3) { scanf("%d", &v);
del(v); }
    } while (ch != 4);
    return 0;
}
```

Output:
```
1.Create(10) → 1.Create(20) → 2.Disp: 20 -
> 10 -> NULL
```

SLIP 7 — Length of Linked List & Infix to Postfix Conversion

Max Marks: 30 (Q1: 10 | Q2: 15 | Viva: 05)

Q.1) Find Length of a Linked List (10M)

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
} *head = NULL;

void insert(int v) {
    struct Node* n = (struct
Node*)malloc(sizeof(struct Node));
    n->data = v; n->next = head; head =
n;
}
int main() {
    int count = 0;
    insert(10); insert(20); insert(30);
    for (struct Node* t = head; t; t = t-
>next) {
        count++;
    }
    printf("Length of Linked List: %d\n",
count);
    return 0;
}
```

Output:
```
Inserted: 10, 20, 30
```

Q.2) Convert Infix Expression to Postfix (15M)

```
#include <stdio.h>
#include <ctype.h>
char st[100]; int top = -1;

void push(char x) { st[++top] = x; }
char pop() { return (top == -1) ? -1 : st[top--];
}
int pri(char x) {
    if (x == '+' || x == '-') return 1;
    if (x == '*' || x == '/') return 2;
    if (x == '^') return 3;
    return 0;
}
int main() {
    char exp[100], *e, x;
    printf("Enter infix: "); scanf("%99s", exp);
    e = exp; printf("Postfix: ");
    while (*e != '\0') {
        if (isalnum(*e)) printf("%c", *e);
        else if (*e == '(') push(*e);
        else if (*e == ')') {
            while ((x = pop()) != '(')
printf("%c", x);
        } else {
            while (top != -1 && pri(st[top]) >=
pri(*e))
                printf("%c", pop());
```

SLIP 8 — Static Queue (Insert/Display) & Doubly Linked List

Max Marks: 30 (Q1: 10 | Q2: 15 | Viva: 05)

Q.1) Static Queue (insert & display) (10M)

```
#include <stdio.h>
#define MAX 5
int q[MAX], f = -1, r = -1;

void insert(int v) {
    if (r == MAX - 1) printf("Queue
Full!\n");
    else {
        if (f == -1) f = 0;
        q[++r] = v;
        printf("Inserted %d\n", v);
    }
}
void display() {
    if (f == -1) printf("Queue Empty\n");
    else {
        printf("Queue: ");
        for (int i = f; i <= r; i++)
printf("%d ", q[i]);
        printf("\n");
    }
}
int main() {
    insert(15); insert(25);
    display();
    return 0;
}
```

```
Output: Inserted 15 | Inserted 25
Queue: 15 25
```

Q.2) Doubly Linked List (Insert, Delete, Display) (15M)

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *p, *n; }
*head = NULL;

void ins(int v) {
    struct Node* t = (struct
Node*)malloc(sizeof(struct Node));
    t->data = v; t->p = NULL; t->n = head;
    if (head) head->p = t;
    head = t;
}
void del(int v) {
    struct Node* t = head;
    while (t && t->data != v) t = t->n;
    if (!t) return;
    if (t->p) t->p->n = t->n; else head = t->n;
    if (t->n) t->n->p = t->p;
    free(t);
}
void disp() {
    for (struct Node* t = head; t; t = t->n)
        printf("%d <-> ", t->data);
    printf("NULL\n");
}
int main() {
    ins(10); ins(20); ins(30); disp();
    del(20); disp();
    return 0;
}
```

```
Output: 30 <-> 20 <-> 10 <-> NULL
After del(20): 30 <-> 10 <-> NULL
```

SLIP 9 — Reverse Singly Linked List & BST (Insert, Search, Preorder)

Max Marks: 30 (Q1: 10 | Q2: 15 | Viva: 05)

Q.1) Reverse a Singly Linked List (10M)

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node*
next; } *head = NULL;

void ins(int v) {
    struct Node* n = (struct
Node*)malloc(sizeof(struct Node));
    n->data = v; n->next = head; head =
n;
}
void reverse() {
    struct Node *p = NULL, *c = head, *n
= NULL;
    while (c) {
        n = c->next; c->next = p;
        p = c; c = n;
    }
    head = p;
}
void disp() {
    for (struct Node* t = head; t; t = t-
>next)
        printf("%d -> ", t->data);
    printf("NULL\n");
}
int main() {
    ins(1); ins(2); ins(3); disp();
    reverse(); disp();
    return 0;
}
```

```
Output: Original: 3 -> 2 -> 1 -> NULL
```

Q.2) BST with Insert, Search & Preorder (15M)

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *l, *r; };

struct Node* ins(struct Node* t, int v) {
    if (!t) {
        t = (struct Node*)malloc(sizeof(struct
Node));
        t->data = v; t->l = t->r = NULL; return t;
    }
    if (v < t->data) t->l = ins(t->l, v);
    else t->r = ins(t->r, v);
    return t;
}
void pre(struct Node* t) {
    if (t) { printf("%d ", t->data); pre(t->l);
pre(t->r); }
}
int search(struct Node* t, int k) {
    if (!t) return 0;
    if (t->data == k) return 1;
    return (k < t->data) ? search(t->l, k) :
search(t->r, k);
}
int main() {
    struct Node* r = NULL;
    r = ins(r, 40); ins(r, 20); ins(r, 60);
    printf("Preorder: "); pre(r);
    printf("\nSearch 20: %s\n", search(r, 20) ?
"Found" : "No");
    return 0;
}
```

SLIP 10 — Merge Sort & Menu-Driven Doubly Circular Linked List

Max Marks: 30 (Q1: 10 | Q2: 15 | Viva: 05)

Q.1) Merge Sort on Initialized Array (10M)

```
#include <stdio.h>

void merge(int a[], int l, int m, int r)
{
    int i = l, j = m + 1, k = 0, b[20];
    while (i <= m && j <= r)
        b[k++] = (a[i] <= a[j]) ? a[i++]
: a[j++];
    while (i <= m) b[k++] = a[i++];
    while (j <= r) b[k++] = a[j++];
    for (i = l, k = 0; i <= r; i++, k++)
a[i] = b[k];
}
void mSort(int a[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        mSort(a, l, m); mSort(a, m + 1,
r);
        merge(a, l, m, r);
    }
}
int main() {
    int A[5] = {87, 30, 44, 20, 51};
    mSort(A, 0, 4);
    printf("Sorted: ");
    for (int i = 0; i < 5; i++)
printf("%d ", A[i]);
    printf("\n");
    return 0;
}
```

Output: Sorted: 20 30 44 51 87

Q.2) Doubly Circular Linked List (Create, Disp, Del Pos) (15M)

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int d; struct Node *p, *n; } *h =
NULL;

void create(int v) {
    struct Node* t = (struct
Node*)malloc(sizeof(struct Node));
    t->d = v;
    if (!h) { h = t->n = t->p = t; }
    else {
        struct Node* l = h->p;
        l->n = t; t->p = l; t->n = h; h->p = t;
    }
}
void delPos(int pos) {
    if (!h) return;
    struct Node* t = h;
    for (int i = 1; i < pos; i++) t = t->n;
    if (t->n == t) { free(t); h = NULL; return; }
    t->p->n = t->n; t->n->p = t->p;
    if (pos == 1) h = t->n;
    free(t);
}
void disp() {
    if (!h) return;
    struct Node* t = h;
    do { printf("%d <-> ", t->d); t = t->n; }
while (t != h);
    printf("(head)\n");
}
int main() {
    create(10); create(20); create(30); disp();
    delPos(2); disp();
    return 0;
}
```

Output: 10 <-> 20 <-> 30 <-> (head) → DelPos(2):
10 <-> 30 <-> (head)

SLIP 11 — QuickSort on Random Numbers & Dynamic Stack

Max Marks: 30 (Q1: 10 | Q2: 15 | Viva: 05)

Q.1) QuickSort on n Random Numbers (10M)

```
#include <stdio.h>
#include <stdlib.h>

void qSort(int a[], int low, int high) {
    if (low < high) {
        int p = a[high], i = low - 1, t;
        for (int j = low; j < high; j++)
{
            if (a[j] < p) {
                i++; t = a[i]; a[i] =
a[j]; a[j] = t;
            }
        }
        t = a[i + 1]; a[i + 1] = a[high];
a[high] = t;
        qSort(a, low, i);
        qSort(a, i + 2, high);
    }
}
int main() {
    int n = 5, a[5];
    printf("Random: ");
    for (int i = 0; i < n; i++) {
        a[i] = rand() % 100; printf("%d
", a[i]);
    }
    qSort(a, 0, n - 1);
```

Q.2) Dynamic Stack (Push, Pop, Search) (15M)

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node* next; } *top
= NULL;

void push(int v) {
    struct Node* n = (struct
Node*)malloc(sizeof(struct Node));
    n->data = v; n->next = top; top = n;
}
void pop() {
    if (!top) { printf("Underflow\n"); return; }
    struct Node* t = top;
    printf("Popped: %d\n", top->data);
    top = top->next; free(t);
}
void search(int k) {
    int pos = 1;
    for (struct Node* t = top; t; t = t->next,
pos++) {
        if (t->data == k) {
            printf("Found %d at pos %d from
top\n", k, pos);
            return;
        }
    }
    printf("Not found\n");
```

SLIP 12 — Polynomial Subtraction & Circular Queue / Graph DFS

Max Marks: 30 (Q1: 10 | Q2: 15 | Viva: 05)

Q.1) Subtraction of Two Polynomials (10M)

```
#include <stdio.h>

int main() {
    int deg, i, p1[10], p2[10];
    printf("Enter max degree: ");
    scanf("%d", &deg);
    printf("Enter Poly 1 coeffs (x^%d to
x^0): ", deg);
    for (i = deg; i >= 0; i--)
scanf("%d", &p1[i]);
    printf("Enter Poly 2 coeffs (x^%d to
x^0): ", deg);
    for (i = deg; i >= 0; i--)
scanf("%d", &p2[i]);

    printf("Subtraction: ");
    for (i = deg; i >= 0; i--) {
        printf("%dx^%d%s", p1[i] - p2[i],
i,
               (i > 0) ? " + " : "\n");
    }
    return 0;
}
```

Output: P1: 3 2 1 | P2: 1 1 1
Subtraction: 2x^2 + 1x^1 + 0x^0

Q.2) Circular Queue (Slip 4) OR Graph DFS Traversal (15M)

```
#include <stdio.h>
int G[10][10], vis[10], n;

void DFS(int v) {
    printf("%d ", v);
    vis[v] = 1;
    for (int j = 0; j < n; j++) {
        if (G[v][j] == 1 && !vis[j]) {
            DFS(j);
        }
    }
}
int main() {
    printf("Enter vertices: "); scanf("%d", &n);
    printf("Enter %dx%d Adjacency Matrix:\n", n,
n);
    for (int i = 0; i < n; i++) {
        vis[i] = 0;
        for (int j = 0; j < n; j++) scanf("%d",
&G[i][j]);
    }
    printf("DFS Traversal starting from 0: ");
    DFS(0);
    printf("\n");
    return 0;
}
```

Output: Vertices: 4 | DFS Traversal starting from 0: 0 1 2 3

SLIP 13 — Merge Sort / SLL Sum & Static Queue with Length()

Max Marks: 30 (Q1: 10 | Q2: 15 | Viva: 05)

Q.1) Merge Sort (Slip 10) / SLL Length & Sum (10M/15M)

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int d; struct Node* n; } *h
= NULL;

void create(int v) {
    struct Node* t = (struct
Node*)malloc(sizeof(struct Node));
    t->d = v; t->n = h; h = t;
}
void stats() {
    int len = 0, sum = 0;
    for (struct Node* t = h; t; t = t->n)
{
        len++; sum += t->d;
    }
    printf("Length: %d | Sum: %d\n", len,
sum);
}
int main() {
    /* For Q.1 Merge Sort, use Slip 10
Q.1 code */
    create(10); create(20); create(30);
    stats();
    return 0;
}
```

Output: Elements: 30, 20, 10
Length: 3 | Sum: 60

Q.2) Static Linear Queue (Insert, Delete, Length) (15M)

```
#include <stdio.h>
#define SZ 5
int q[SZ], f = -1, r = -1;

void insert(int v) {
    if (r == SZ - 1) printf("Overflow\n");
    else {
        if (f == -1) f = 0;
        q[++r] = v;
    }
}
void del() {
    if (f == -1 || f > r) printf("Underflow\n");
    else {
        printf("Deleted: %d\n", q[f++]);
        if (f > r) f = r = -1;
    }
}
void length() {
    int len = (f == -1 || f > r) ? 0 : (r - f +
1);
    printf("Total elements in queue: %d\n", len);
}
int main() {
    insert(5); insert(10); insert(15);
    length(); del(); length();
    return 0;
}
```

Output: Total elements: 3 → Deleted: 5 → Total elements: 2

SLIP 14 — Recursive Binary Search & BST (Total & Odd Count)

Max Marks: 30 (Q1: 10 | Q2: 15 | Viva: 05)

Q.1) Recursive Binary Search on Char Array (10M)

```
#include <stdio.h>

int bSearch(char a[], int l, int h, char
k) {
    if (h >= l) {
        int m = (l + h) / 2;
        if (a[m] == k) return m;
        if (a[m] > k) return bSearch(a,
l, m - 1, k);
        return bSearch(a, m + 1, h, k);
    }
    return -1;
}
int main() {
    char a[] = {'a', 'e', 'i', 'o', 'u'},
k;
    printf("Enter char to search
(a/e/i/o/u): ");
    scanf(" %c", &k);
    int res = bSearch(a, 0, 4, k);
    if (res != -1) printf("Found at index
%d\n", res);
    else printf("Not found\n");
    return 0;
}
```

Output: Enter char: o → Found at index 3

Q.2) BST (In-Order, Total Nodes, Odd Count) (15M)

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int d; struct Node *l, *r; };

struct Node* ins(struct Node* t, int v) {
    if (!t) {
        t = (struct Node*)malloc(sizeof(struct
Node));
        t->d = v; t->l = t->r = NULL; return t;
    }
    if (v < t->d) t->l = ins(t->l, v);
    else t->r = ins(t->r, v);
    return t;
}
void in(struct Node* t) {
    if (t) { in(t->l); printf("%d ", t->d); in(t-
>r); }
}
int total(struct Node* t) {
    return t ? (1 + total(t->l) + total(t->r)) :
0;
}
int odd(struct Node* t) {
    if (!t) return 0;
    return (t->d % 2 != 0) + odd(t->l) + odd(t-
>r);
}
int main() {
    struct Node* r = NULL;
    r = ins(r, 15); ins(r, 10); ins(r, 20); ins(r,
7);
    printf("In-order: "); in(r);
    printf("\nTotal: %d | Odd: %d\n", total(r),
odd(r));
    return 0;
}
```

Output: In-order: 7 10 15 20 | Total: 4 | Odd: 2

SLIP 15 — Quick Sort on n Numbers & Dynamic Queue

Max Marks: 30 (Q1: 10 | Q2: 15 | Viva: 05)

Q.1) Sort n Numbers using Quick Sort (10M)

```
#include <stdio.h>

void qSort(int a[], int low, int high) {
    if (low < high) {
        int p = a[high], i = low - 1, t;
        for (int j = low; j < high; j++)
{
            if (a[j] < p) {
                i++; t = a[i]; a[i] =
a[j]; a[j] = t;
            }
        }
        t = a[i + 1]; a[i + 1] = a[high];
a[high] = t;
        qSort(a, low, i); qSort(a, i + 2,
high);
    }
}
int main() {
    int n, a[50];
    printf("Enter n: "); scanf("%d", &n);
    for (int i = 0; i < n; i++)
scanf("%d", &a[i]);
    qSort(a, 0, n - 1);
    printf("Sorted: ");
    for (int i = 0; i < n; i++)
printf("%d ", a[i]);
    return 0;
```

Q.2) Dynamic Queue (Insert, Delete, Display) (15M)

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int d; struct Node* n; } *f = NULL,
*r = NULL;

void ins(int v) {
    struct Node* t = (struct
Node*)malloc(sizeof(struct Node));
    t->d = v; t->n = NULL;
    if (!r) f = r = t;
    else { r->n = t; r = t; }
}
void del() {
    if (!f) { printf("Underflow\n"); return; }
    struct Node* t = f;
    printf("Deleted: %d\n", f->d);
    f = f->n; if (!f) r = NULL;
    free(t);
}
void disp() {
    for (struct Node* t = f; t; t = t->n)
        printf("%d -> ", t->d);
    printf("NULL\n");
}
int main() {
    ins(100); ins(200); disp();
    del(); disp();
    return 0;
```

SLIP 16 — Character Array Bubble Sort & Dynamic Circular Queue

Max Marks: 30 (Q1: 10 | Q2: 15 | Viva: 05)

Q.1) Sort Character Array using Bubble Sort (10M)

```
#include <stdio.h>

int main() {
    char a[] = {'z', 'a', 'm', 'b', 'k'},
t;
    int n = 5, i, j;

    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                t = a[j];
                a[j] = a[j + 1];
                a[j + 1] = t;
            }
        }
    }
    printf("Sorted characters: ");
    for (i = 0; i < n; i++) printf("%c ",
a[i]);
    printf("\n");
    return 0;
}
```

Output: Input: z a m b k
Sorted characters: a b k m z

Q.2) Dynamic Circular Queue (Insert, Delete, Display) (15M)

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int d; struct Node* n; } *f = NULL,
*r = NULL;

void ins(int v) {
    struct Node* t = (struct
Node*)malloc(sizeof(struct Node));
    t->d = v;
    if (!f) f = r = t;
    else { r->n = t; r = t; }
    r->n = f;
}
void del() {
    if (!f) return;
    if (f == r) { free(f); f = r = NULL; }
    else {
        struct Node* t = f;
        f = f->n; r->n = f; free(t);
    }
}
void disp() {
    if (!f) return;
    struct Node* t = f;
    do { printf("%d -> ", t->d); t = t->n; } while
(t != f);
    printf("(front)\n");
}
int main() {
    ins(10); ins(20); ins(30); disp();
    del(); disp();
    return 0;
}
```

Output: 10 -> 20 -> 30 -> (front) → After del: 20 -> 30 -> (front)

SLIP 17 — Structure Bubble Sort (Customer) & Balanced Parentheses

Max Marks: 30 (Q1: 10 | Q2: 15 | Viva: 05)

Q.1) Sort Customer Structure on custid (10M)

```
#include <stdio.h>
struct Customer {
    int custid; char name[20], addr[30];
};
int main() {
    struct Customer c[3] = {
        {3, "Amit", "Pune"},
        {1, "Rohan", "Mumbai"},
        {2, "Neha", "Nashik"}
    }, t;
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2 - i; j++)
            if (c[j].custid > c[j +
1].custid) {
                t = c[j]; c[j] = c[j +
1]; c[j + 1] = t;
            }
    for (int i = 0; i < 3; i++)
        printf("%d %s %s\n", c[i].custid,
c[i].name, c[i].addr);
    return 0;
}
```

Output: 1 Rohan Mumbai | 2 Neha Nashik | 3 Amit Pune

Q.2) Check Balanced Parentheses using Stack (15M)

```
#include <stdio.h>
int main() {
    char exp[100], st[100];
    int top = -1, ok = 1;
    printf("Enter expression: "); scanf("%99s",
exp);
    for (int i = 0; exp[i] != '\0'; i++) {
        char c = exp[i];
        if (c == '(' || c == '{' || c == '[')
st[++top] = c;
        else if (c == ')' || c == '}' || c == ']')
{
            if (top == -1) { ok = 0; break; }
            char o = st[top--];
            if ((c == ')' && o != '(') ||
                (c == '}' && o != '{') ||
                (c == ']' && o != '[')) { ok = 0;
break; }
        }
    }
    if (top != -1) ok = 0;
    printf("%s\n", ok ? "BALANCED" : "NOT
BALANCED");
    return 0;
}
```

Output: Input: (a+b)*(c-d) → BALANCED

SLIP 18 — Linear Search on City Structure & Dynamic Queue Search

Max Marks: 30 (Q1: 10 | Q2: 15 | Viva: 05)

Q.1) Search City Code by City Name (10M)

```
#include <stdio.h>
#include <string.h>

struct city {
    int city_code;
    char name[30];
};
int main() {
    struct city c[3] = {
        {101, "Pune"}, {102, "Mumbai"},
{103, "Nagpur"}
    };
    char key[30];
    printf("Enter city name: ");
    scanf("%29s", key);
    for (int i = 0; i < 3; i++) {
        if (strcmp(c[i].name, key) == 0)
{
            printf("City Code: %d\n",
c[i].city_code);
            return 0;
        }
    }
    printf("City not found.\n");
    return 0;
}
```

Output: Enter city name: Mumbai → City Code: 102

Q.2) Dynamic Queue (Insert, Length, Search) (15M)

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int d; struct Node* n; } *f = NULL,
*r = NULL;

void ins(int v) {
    struct Node* t = (struct
Node*)malloc(sizeof(struct Node));
    t->d = v; t->n = NULL;
    if (!r) f = r = t; else { r->n = t; r = t; }
}
void length() {
    int c = 0;
    for (struct Node* t = f; t; t = t->n) c++;
    printf("Length: %d\n", c);
}
void search(int k) {
    int p = 1;
    for (struct Node* t = f; t; t = t->n, p++)
        if (t->d == k) { printf("Found %d at pos
%d\n", k, p); return; }
    printf("Not found\n");
}
int main() {
    ins(10); ins(20); ins(30);
    length(); search(20);
    return 0;
}
```

Output: Inserted 10, 20, 30 → Length: 3 → Found 20 at pos 2

SLIP 19 — Singly Linked List of Vowels & Character BST

Max Marks: 30 (Q1: 10 | Q2: 15 | Viva: 05)

Q.1) Singly Linked List of Vowels (10M)

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    char v;
    struct Node* next;
} *head = NULL;

void ins(char c) {
    struct Node* n = (struct
Node*)malloc(sizeof(struct Node));
    n->v = c; n->next = head; head = n;
}
int main() {
    ins('u'); ins('o'); ins('i');
ins('e'); ins('a');
    printf("Vowels List: ");
    for (struct Node* t = head; t; t = t-
>next)
        printf("%c -> ", t->v);
    printf("NULL\n");
    return 0;
}
```

Output: Vowels List: a -> e -> i -> o -> u -> NULL

Q.2) BST of Char Data (Display, Leaf & Non-Leaf) (15M)

```
#include <stdio.h>
#include <stdlib.h>
struct Node { char d; struct Node *l, *r; };

struct Node* ins(struct Node* t, char v) {
    if (!t) {
        t = (struct Node*)malloc(sizeof(struct
Node));
        t->d = v; t->l = t->r = NULL; return t;
    }
    if (v < t->d) t->l = ins(t->l, v);
    else if (v > t->d) t->r = ins(t->r, v);
    return t;
}
void in(struct Node* t) {
    if (t) { in(t->l); printf("%c ", t->d); in(t-
>r); }
}
int leaf(struct Node* t) {
    if (!t) return 0;
    return (!t->l && !t->r) ? 1 : leaf(t->l) +
leaf(t->r);
}
int nonLeaf(struct Node* t) {
    if (!t || (!t->l && !t->r)) return 0;
    return 1 + nonLeaf(t->l) + nonLeaf(t->r);
}
int main() {
    struct Node* r = NULL;
    char s[] = {'M', 'D', 'T', 'A', 'G'};
    for (int i = 0; i < 5; i++) r = ins(r, s[i]);
    in(r); printf("\nLeaf:%d NonLeaf:%d\n",
leaf(r), nonLeaf(r));
    return 0;
}
```

SLIP 20 & GRAPH OR QUESTIONS — Insertion Sort, BST, BFS & Degree

CA-202-MJP Complete Reference

Slip 20 Q.1 & Q.2 Cross-Reference

- **Slip 20 Q.1) Insertion Sort (10M):** Use exact code from **Slip 3 Q.1** (Page 2).
- **Slip 20 Q.2) BST Ascending Order (15M):** In-order traversal of BST prints data in ascending order. Use **Slip 5 Q.2** (Page 3).
- **Slip 20 Q.2 OR) Infix to Postfix (15M):** Use exact code from **Slip 7 Q.2** (Page 4).

Graph BFS Output (Right):

```
V0: In=0 Out=2 | V1: In=1 Out=1 | BFS: 0 1 2
```

Slip 4 / 14 OR: Graph BFS, Indegree & Outdegree (15M)

```
#include <stdio.h>
int main() {
    int n = 3, G[3][3] = {{0,1,1},{0,0,1},
{0,0,0}};
    int q[10], f = 0, r = 0, vis[3] = {0};
    for (int i = 0; i < n; i++) {
        int in = 0, out = 0;
        for (int j = 0; j < n; j++) { in += G[j]
[i]; out += G[i][j]; }
        printf("V%d: In=%d Out=%d | ", i, in,
out);
    }
    printf("\nBFS: ");
    q[r++] = 0; vis[0] = 1;
    while (f < r) {
        int u = q[f++]; printf("%d ", u);
        for (int v = 0; v < n; v++)
            if (G[u][v] && !vis[v]) { vis[v] = 1;
q[r++] = v; }
    }
    return 0;
}
```

PART 2: ELS-242-MNP (Slips 5 & 6) — 7-Bit Hamming Code Simulation in Dev-C++

Data Communication Lab

7-Bit Hamming Word Layout (Even Parity):

Bit 1 (P1)	Bit 2 (P2)	Bit 3 (D3)	Bit 4 (P4)	Bit 5 (D5)	Bit 6 (D6)	Bit 7 (D7)
D3 ⊕ D5 ⊕ D7	D3 ⊕ D6 ⊕ D7	Data Bit 1 (LSB)	D5 ⊕ D6 ⊕ D7	Data Bit 2	Data Bit 3	Data Bit 4 (MSB)

ELS Slip 5: 7-Bit Hamming Code Generation

```
#include <stdio.h>

int main() {
    int d[8], i;
    printf("Enter 4-bit data (D7 D6 D5
D3): ");
    scanf("%d %d %d %d", &d[7], &d[6],
&d[5], &d[3]);

    /* Even Parity Calculation using XOR
*/
    d[1] = d[3] ^ d[5] ^ d[7];
    d[2] = d[3] ^ d[6] ^ d[7];
    d[4] = d[5] ^ d[6] ^ d[7];

    printf("Generated 7-Bit Hamming Code:
");
    for (i = 7; i >= 1; i--) printf("%d
", d[i]);
    printf("\n");
    return 0;
}
```

Output: Data (D7 D6 D5 D3): 1 0 1 1
Generated 7-Bit Code: 1 0 1 0 1 0 1

ELS Slip 6: Error Detection & Correction

```
#include <stdio.h>

int main() {
    int r[8], i, c1, c2, c4, err;
    printf("Enter received 7 bits (bit 7 to
1):\n");
    for (i = 7; i >= 1; i--) scanf("%d", &r[i]);

    c1 = r[1] ^ r[3] ^ r[5] ^ r[7];
    c2 = r[2] ^ r[3] ^ r[6] ^ r[7];
    c4 = r[4] ^ r[5] ^ r[6] ^ r[7];
    err = c4 * 4 + c2 * 2 + c1;

    if (err == 0) printf("No error detected.\n");
    else {
        printf("Error at bit position: %d\n",
err);
        r[err] = !r[err];
        printf("Corrected Code: ");
        for (i = 7; i >= 1; i--) printf("%d ",
r[i]);
    }
    return 0;
}
```

Output: Received: 1 0 0 0 1 0 1
Error at pos: 5 → Corrected: 1 0 1 0 1 0 1